

Table Format Optimizations in Managed Spark for Dataproc

Isha Tarte
Google, USA
ishatarte@google.com

Vishal Karve
Google, India
vkarve@google.com

Rajarshi Sarkar
Google, India
rajarshisarkar@google.com

Zihan Cao
Google, USA
zihanco@google.com

Jayadeep Jayaraman
Google, India
jjayadeep@google.com

Jingwei Lu
Google, USA
jingweilu@google.com

Aditya Shah
Google, USA
adityabshah@google.com

Huadong Liu
Google, USA
huadongliu@google.com

Wei Yan*
OpenAI, USA
ywskycn@gmail.com

Abhishek Modi
Google, India
abmod@google.com

Sourabh Badhya
Google, India
sourabhsbadhya@google.com

Haymant Mangla
Google, India
haymant@google.com

Warren Zhu
Google, USA
warrenzhu@google.com

Abstract

Efficient data lifecycle management is critical for modern Lakehouse architectures, yet standard open source implementations of transactional updates and storage maintenance often suffer from prohibitive I/O and compute costs. This paper presents novel, format-aware optimizations implemented within the Dataproc Spark engine, including workload aware optimizations. For transactional workloads on Lakehouse, we introduce Precision Merge, which utilizes a row-level tracking mechanism to minimize shuffle and wasted compute. Furthermore, we propose significant enhancements to table maintenance operations, utilizing Direct Binary Merge for efficient compaction and a multi-tiered, asynchronous framework for VACUUM processes. Extensive benchmarking demonstrates that these combined strategies deliver over a 50% reduction in execution time, a 10x decrease in shuffle volume, and up to a 10x speedup in routine maintenance, providing a highly scalable and cost-effective solution for high-throughput Lakehouse management.

CCS Concepts

• Information systems → Data management systems.

Keywords

Lakehouse Architectures, Apache Spark, Apache Iceberg, Delta Lake, Data Lifecycle Management, Table Maintenance

*Work done while at Google.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

FORMATS '26, Bengaluru, India

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2635-4/2026/05

<https://doi.org/10.1145/3802514.3809179>

ACM Reference Format:

Isha Tarte, Jayadeep Jayaraman, Abhishek Modi, Vishal Karve, Jingwei Lu, Sourabh Badhya, Rajarshi Sarkar, Aditya Shah, Haymant Mangla, Zihan Cao, Huadong Liu, Warren Zhu, and Wei Yan. 2026. Table Format Optimizations in Managed Spark for Dataproc. In *1st International Workshop on Data FORMATS for Modern Architectures and Workloads (FORMATS '26)*, May 31-June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3802514.3809179>

1 Introduction

Lakehouse architectures have emerged as an important paradigm in the world of unified analytics, combining the rigorous data management of enterprise data warehouses with the petabyte-scale, low-cost scalability of data lakes [5]. However, efficient data lifecycle management—encompassing both transactional updates (such as MERGE) and vital maintenance routines (such as file compaction and snapshot cleanup) frequently becomes a severe performance bottleneck [7, 9]. The continuous maintenance required to uphold atomic, consistent, isolated, and durable (ACID) transactional guarantees at petabyte scale regularly degrades into the dominant consumer of distributed cluster resources [7]. This degradation is driven by massive, often unnecessary data shuffling during execution [8], huge volumes of data read from and written to external cloud object stores [4], and extreme metadata processing overhead associated with fragmented object tracking [10].

While industry solutions have introduced Low-Shuffle Merge to mitigate these issues [6, 8], standard open-source implementations still typically rely on generic join strategies, such as Outer Joins, that force full-table shuffles even when only a small fraction of rows are modified [1, 2]. Furthermore, routine compaction tasks often incur significant CPU overhead by deserializing and re-serializing immutable data blocks [1, 2]. In cloud environments, these inefficiencies translate directly into inflated compute costs and increased latency.

To address these challenges, this paper introduces improved Spark plan optimizations and low-level file manipulation techniques implemented within Lightning Engine for Spark. We detail architectural improvements for Delta Lake and Apache Iceberg, focusing specifically on the reduction of execution times, memory pressure, and I/O throughput. To the best of our knowledge, the specific suite of optimizations proposed below has not yet been integrated into standard open-source architectures. Section 2.1 presents the **Workload Aware Optimizations**; Section 2.2 details the optimizations done in **MERGE workloads**; and Section 2.3 covers optimization techniques in **Table Maintenance operations** such as **OPTIMIZE** and **VACUUM**.

2 Optimization Strategies

2.1 Workload Aware Optimizations

2.1.1 Workload-Aware Data Skipping and Schema Pruning for Joins. Standard read operations frequently build table scans by accessing all files and only use conditions with a constant. Additional join conditions are typically not pushed down to the storage layer at plan time [1, 2]. To address this, we introduce format-aware optimizations that leverage runtime source statistics and schema-aware projections to transform full-table scans into targeted, relevant-only I/O operations.

- **Metadata-Driven Statistics Collection:** The engine retrieves minimum, maximum, distinct and null values directly from the transaction log for deterministic sources or captures them during materialization phases to avoid secondary job overhead. These statistics can then be used to filter table scans involving join operations.
- **Dynamic Data Filtering:** Runtime statistics are injected as dynamic boundary filters into the table scan plan, allowing the engine to skip files or partitions that do not overlap with the source data range during MERGE operations.
- **Adaptive Partition Gap Skipping:** For high-cardinality partition columns with sparse updates, the engine generates discrete predicates to precisely target required partitions while skipping massive volumes of untouched data.

These optimizations are most effective for localized DML operations, such as clustered updates or sparse updates on high-cardinality columns where large data gaps can be skipped. They are particularly beneficial for wide-table joins and massive-scale ingestions where materialization can gather statistics for pruning without additional overhead. Extensive benchmarking demonstrates that these strategies yield performance gains exceeding 50% and a 10x reduction in shuffle volume. Specifically, source-injected pruning reduces target scanned bytes by up to 99.8%, while write-through materialization halves execution latency for massive-scale datasets.

2.1.2 File Layout Optimization and Split Size Estimation. Parquet files with Iceberg catalog can be optimized to improve query performance dramatically by implementing file layout optimizations such as sorting, clustering, row group sizing etc. The engine monitors historical job runs to identify frequently used join columns, split sizes etc, and uses these to perform file layout optimizations such as identifying columns for clustering and file size estimation.

Most real world tables and schemas have lots of columns with variable data types; due to the variability in length of such columns, Spark by default uses some naive techniques to estimate splits and partitions which often times leads to over-partitioning [1]. We introduce a Footer-Based Split Estimation mechanism. Unlike rigid workaround solutions that simply adjust maxPartitionBytes [1], our approach dynamically calibrates split sizes based on the actual density of the projected columns.

- **Statistical Projection:** The system extracts granular metadata at write time, specifically the uncompressed data size and total row counts.
- **Metadata Sampling:** Before the Scan stage, the driver performs a read of Parquet footers from a randomized set of partition files.
- **Factor Calculation:** Using data collected either via Statistical projection or Metadata sampling, the engine calculates an average size per row for the requested schema and using the uncompressed size and average row length the engine estimates the true in-memory size of the input split, allowing the scheduler to pack more data into fewer tasks when only a small subset of data regardless of fixed or variable length is required.

This optimization improved performance by 10% for a customer's workloads, particularly those that read only a small subset of columns from the overall table schema.

2.1.3 Intelligent Maintenance via Catalog. The engine proactively captures historical runs to analyze the frequency and duration of previous runs. In addition, it takes into account the amount of data being written in the current run. By evaluating the history and information from the current run, the system determines if the next write operation is likely to occur far enough in the future or if significant data has been modified in the current run, then it automatically triggers an OPTIMIZE operation. This ensures that compaction and maintenance leverage idle cluster time, preventing maintenance tasks from interfering with active, high-throughput ingestions. Similarly, the engine also keeps a track of the amount of data that can be cleaned via the Vacuum operation and triggers vacuum to keep the size of the table manageable.

2.2 Precision MERGE

Standard MERGE for table formats like Delta Lake and Apache Iceberg follows a multi-phase algorithm. This involves a join to identify touched files, followed by a join of the source table with all data within those touched files [1]. The join type is decided based on MERGE clauses but involves all touched files and scans all the columns regardless of whether they will be used [2]. If the number of rows that will remain unchanged in these touched files is large, shuffling them is mostly wasteful since they merely need to be rewritten into new files. We propose Precision Merge as a mechanism to decouple row identification from data rewriting, thereby minimizing write amplification and wasted compute. Precision Merge introduces a row-level tracking mechanism alongside dynamic join reduction and only projecting the required metadata fields from the target table.

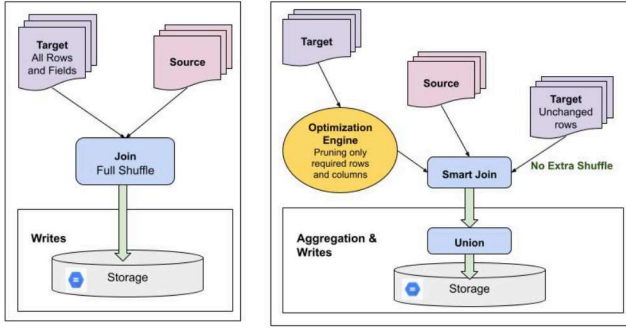


Figure 1: Improved spark plan with Precision MERGE

- **Bitmap Indexing:** During the initial phase to find touched files, the engine identifies not just the files, but also the exact row positions of the matched rows in each file using the file metadata which are stored in a memory efficient bitmap. The write phase utilizes the bitmap to isolate only the specific rows requiring updates or deletions, performing the join operation exclusively on this subset.
- **Handling Delete clauses:** This clause involves a full scan of the target for identifying even the rows which are not matching. This implies that files are not filtered during the findTouchedFiles stage. We introduce an optimization where we handle this clause separately, the relevant rows are processed via a filter rather than a join. This allows precision merge to function even with delete clauses, reducing shuffle volume by treating the operation as a metadata-guided filter.
- **Schema-Aware Column Pruning:** The engine reduces I/O throughput by projecting only the columns required for specific operations, such as projecting only join keys for deletions or skipping payload reads for fully overwritten rows.
- **Dynamic Join Reduction:** We only use outer join when inserts are present and the target table is never involved in outer join. This allows for faster Broadcast Hash Joins in cases of smaller target or source.

In practical production environments, MERGE operations predominantly involve updating massive target tables with small, incremental batches of new data. To ensure our evaluation of Precision Merge reflects these real-world conditions, we designed a representative benchmark derived from the TPC-DS dataset. The experimental setup utilized a 27-node cluster—with each worker node provisioned with 16 vCPUs, 64 GB RAM, and 2 local NVMe SSDs, using open-source Delta Lake as our baseline for speedup comparisons. The workload targeted a 1TB partitioned store_sales table and evaluated a matrix of data change scenarios. By varying the source table overlap to test 1%, 10%, and 50% matched rows, we accurately simulated the spectrum of real-world operations, from highly sparse, incremental updates to denser merges. As illustrated in Figure 2, Precision Merge yields the most significant speedups during these highly sparse updates.

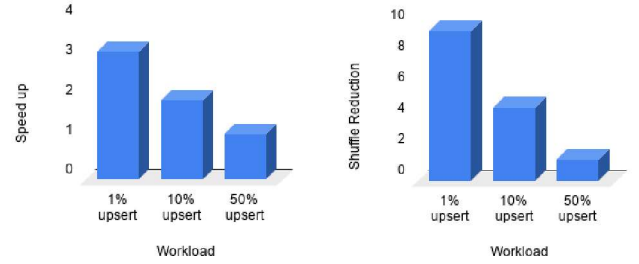


Figure 2: Speedup and Shuffle Data Reduction with Precision MERGE

2.3 Optimizations for Table Maintenance Operations

2.3.1 Compaction. Iceberg compaction currently requires a lot of fine tuning and user input to run optimally on a cluster [3]. The current implementation of Optimize command in Delta runs a Spark job per partition and target file size [2]. We implemented the following techniques to intelligently execute compaction.

- **Parallelism based on available cluster resources:** A cluster aware and partition job aware implementation to determine the number of jobs that can run in parallel to coalesce files. This greatly improves the runtime performance for partitioned cases by 95%.
- **Partition Awareness:** The compaction job respects partition schemes of the table, effectively eliminating the need for data shuffling during execution. Furthermore, by consolidating the processing of multiple partitions into a single job, the architectural design significantly mitigates memory overhead on the driver node. Empirically, this optimization yielded a 50% reduction in driver memory consumption.
- **Direct Binary Merge:** To eliminate the overhead of record materialization, we propose a Parquet Row-Group Append strategy which combines compressed row groups directly without deserializing the underlying data rows. In standard approaches, the schema checks are handled on the Spark driver whereas we leverage the Spark executor to perform schema checks [10]. This improves the file opening cost by 50%. We also copy the additional metadata of every row group like bloom filters, column index and offset index. In scenarios involving file sizes of 128MB on a 1.25 TB table, we were able to see 45-55% runtime improvement on compaction using this method.

To systematically evaluate the combined performance and resource efficiency of these three optimizations during the rewrite data files procedure, we established a benchmark suite comparing our engine against a baseline OSS implementation. The evaluation utilized the TPC-DS store_sales table across varied workloads on a Dataproc cluster consisting of 27 worker nodes equipped with hyperdisk-balanced boot disks. The benchmark includes 1.25TB batch tables (compacting 128MB files to 512MB) and 100GB streaming tables (compacting 4MB files to 256MB). The resulting execution speedups for these compaction workloads are illustrated in Figure 3.

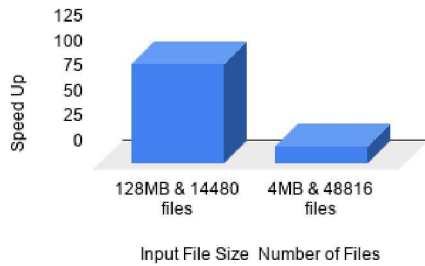


Figure 3: Speedup with improved OPTIMIZE in Iceberg

2.3.2 VACUUM. To mitigate the I/O bottlenecks inherent in Apache Iceberg’s `expire_snapshots` operations and Delta Vacuum [1, 2], we introduce a multi-tiered, asynchronous parallel framework designed to replace legacy sequential deletion processes. This framework significantly enhances throughput by eliminating storage-layer concurrency limits and shifting from blocking I/O models to an asynchronous event-driven architecture. The performance improvements are driven by four targeted optimizations at the execution and I/O layers:

- **Concurrent Batch Processing:** Legacy implementations processed file deletions sequentially on a single thread. The optimized framework dispatches partitioned file batches to a shared, managed thread pool, allowing multiple batches to be processed simultaneously.
- **Asynchronous I/O Execution:** To prevent blocking during synchronous storage requests, the framework leverages Java’s `CompletableFuture` to map file batches to independent asynchronous tasks.
- **Parallel File-Type Erasure:** The system eliminates sequential dependencies between different metadata layers. Rather than waiting for all data files to be deleted before addressing manifest files, the framework allows data, manifest, and delete files to be erased concurrently, thereby effectively overlapping I/O latency.

We evaluated the performance of the VACUUM operation TPC-DS tables. To reflect real-world cost and performance demands, our benchmark specifically isolated and measured the metadata processing overhead associated with deleting stale, unreferenced files. The experimental infrastructure utilized a Dataproc cluster configured with one master and 27 worker nodes. To optimize I/O, each node was provisioned with 16 cores, 64GB memory, a 500GB primary disk, and 2 SSDs. Our testing focused on the overhead of deleting 10,000 and 100,000 files from an un-partitioned table. Because file deletion is fundamentally a metadata operation, the physical file size was kept intentionally small. The speed-up obtained for delta and iceberg is shown in Fig 4.

3 Conclusion and Future Work

Optimizing Lakehouse operations requires a workload-aware, multi-layered approach that addresses both plan and format-level inefficiencies. Our evaluation demonstrates that Precision Merge achieves a 50% reduction in execution time, while achieving a 10x

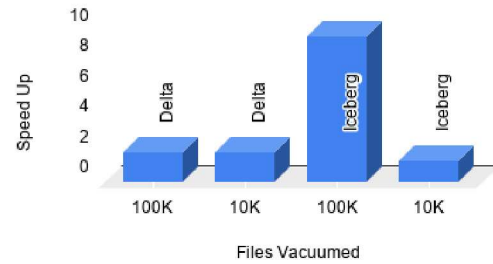


Figure 4: Speedup with improved VACUUM in Delta & Iceberg

speed up in table maintenance operations. These strategies provide critical performance differentiators in Spark Lightning Engine, aligning with modern demands for I/O efficiency. Future work will focus on integrating these format-aware optimizations into the Gluten and Velox framework to leverage native execution for further performance gains. We intend to enhance data pruning by propagating granular metadata statistics from Delta and Iceberg tables directly to the Spark optimizer, allowing for more intelligent plan-level filtering. Additionally, we aim to implement shuffle-free VACUUM operations in Delta Lake and increase the scope of binary MERGE to include stripe appending for ORC formats.

4 Acknowledgements

We extend our sincere gratitude to Newton Alex, Sam McVeety, and Pavan Edara for their invaluable guidance, continued support, and comprehensive review during the submission process.

References

- [1] 2026. Apache Spark. <https://github.com/apache/spark>.
- [2] 2026. Delta Lake. <https://github.com/delta-io/delta>.
- [3] Alex Merced. 2022. *Compaction in Apache Iceberg: Fine-Tuning Your Iceberg Table’s Data Files*. <https://www.dremio.com/blog/compaction-in-apache-iceberg-fine-tuning-your-iceberg-tables-data-files/>
- [4] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman Van Hovell, Adrian Ionescu, Alicja Luszczak, et al. 2020. Delta lake: high-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3411–3424.
- [5] Michael Armbrust, Ali Ghodsi, Reynold Xin, Matei Zaharia, et al. 2021. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of CIDR*, Vol. 8. sn, 28.
- [6] Himanshu Raja Justin Breese Tom van Bussel Lars Kroll Sabir Akhadev Tathagata Das Bart Samwel, Piyush Revuri and Prakhar Jain. 2022. *Faster MERGE Performance With Low-Shuffle MERGE and Photon*. <https://www.databricks.com/blog/faster-merge-performance-low-shuffle-merge-and-photon>
- [7] Anja Gruenheid, Jesús Camacho-Rodríguez, Carlo Curino, Raghu Ramakrishnan, Stanislav Pak, Sumedh Sakdeo, Lenisha Gandhi, Sandeep K Singhal, Pooja Nilangekar, and Daniel J Abadi. 2025. AutoComp: Automated data compaction for log-structured tables in data lakes. In *Companion of the 2025 International Conference on Management of Data*. 404–417.
- [8] Microsoft. 2026. *Low Shuffle Merge optimization on Delta tables - Azure Synapse Analytics*. <https://learn.microsoft.com/en-us/azure/synapse-analytics/spark/low-shuffle-merge-for-apache-spark>
- [9] Anton Okolnychyi, Chao Sun, Kazuyuki Tanimura, Russell Spitzer, Ryan Blue, SzeHon Ho, Yufei Gu, Vishwanath Lakkundi, and DB Tsai. 2024. Petabyte-scale row-level operations in data lakehouses. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4159–4172.
- [10] Xinli Shang. 2025. *Pull Request #14435: Apache Iceberg*. <https://github.com/apache/iceberg/pull/14435>